

Chapter 11 - Fine-Tuning Representation Models for Classification

Supervised Classification

Recall that in chapter 4, we finetuned **pretrained models** to perform specific tasks (sentiment prediction). In that chapter, we kept the pretrained model frozen (nontrainable), not updating its weights and just fine-tuning the classification head.

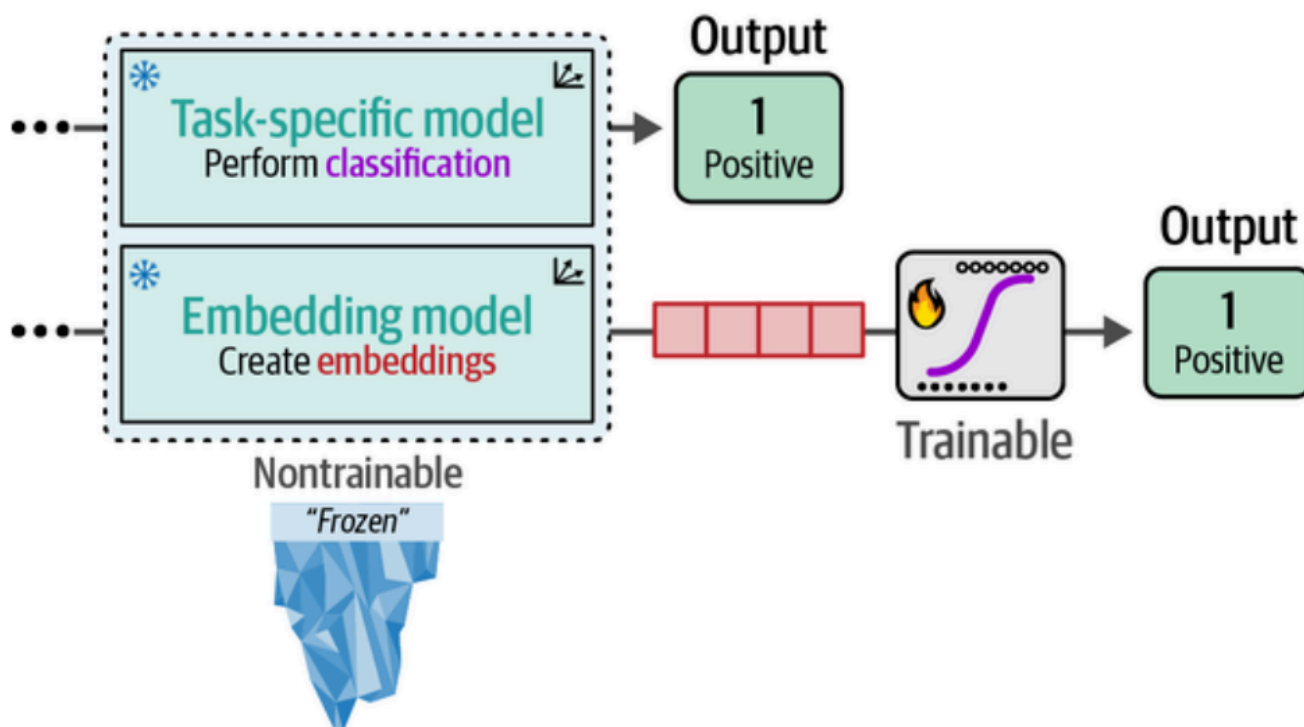


Figure 11-1. In *Chapter 4*, we used pretrained models to perform classification without updating their weight. These models were kept “frozen.”

In this section, we still fine-tune the model, but we keep them trainable. In chapter 4, we trained a mapping of the embedding model output to classification, without touching the pretrained model.

Here, we will treat the pretrained model and the classification head as one architecture during training, allowing the pretrained model to change its weights and parameters.

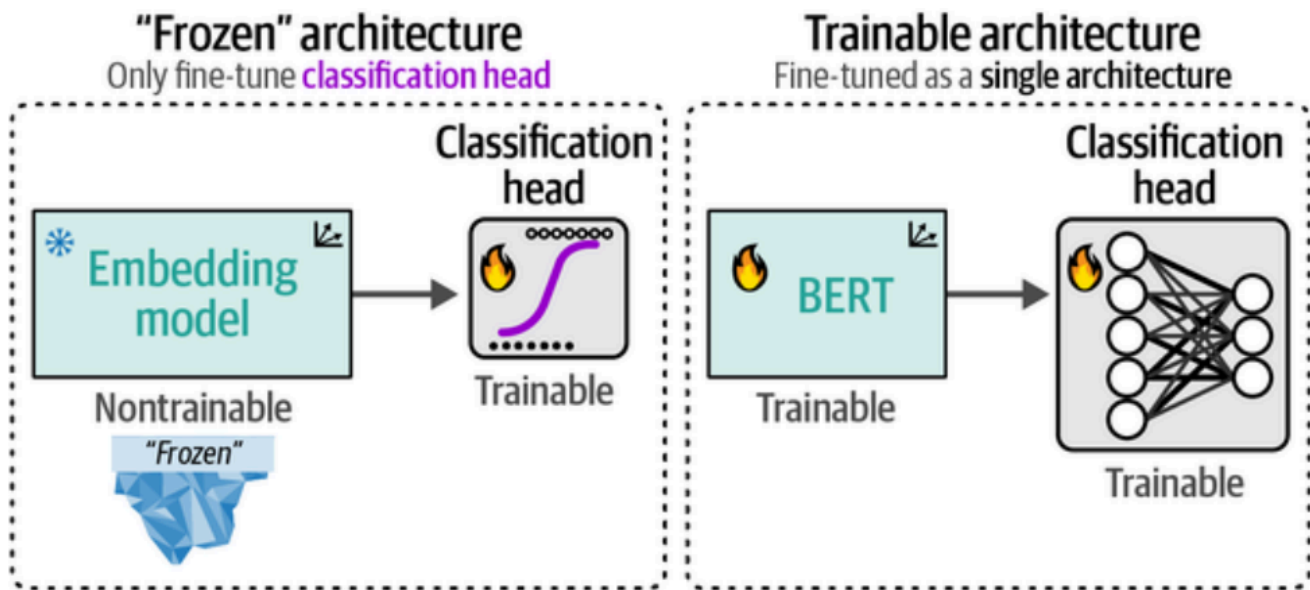


Figure 11-2. Compared to the “frozen” architecture, we instead train both the pretrained BERT model and the classification head. A backward pass will start at the classification head and go through BERT.

We will use the `bert-base-cased` model as our pretrained model and train it with a classification head (neural network).

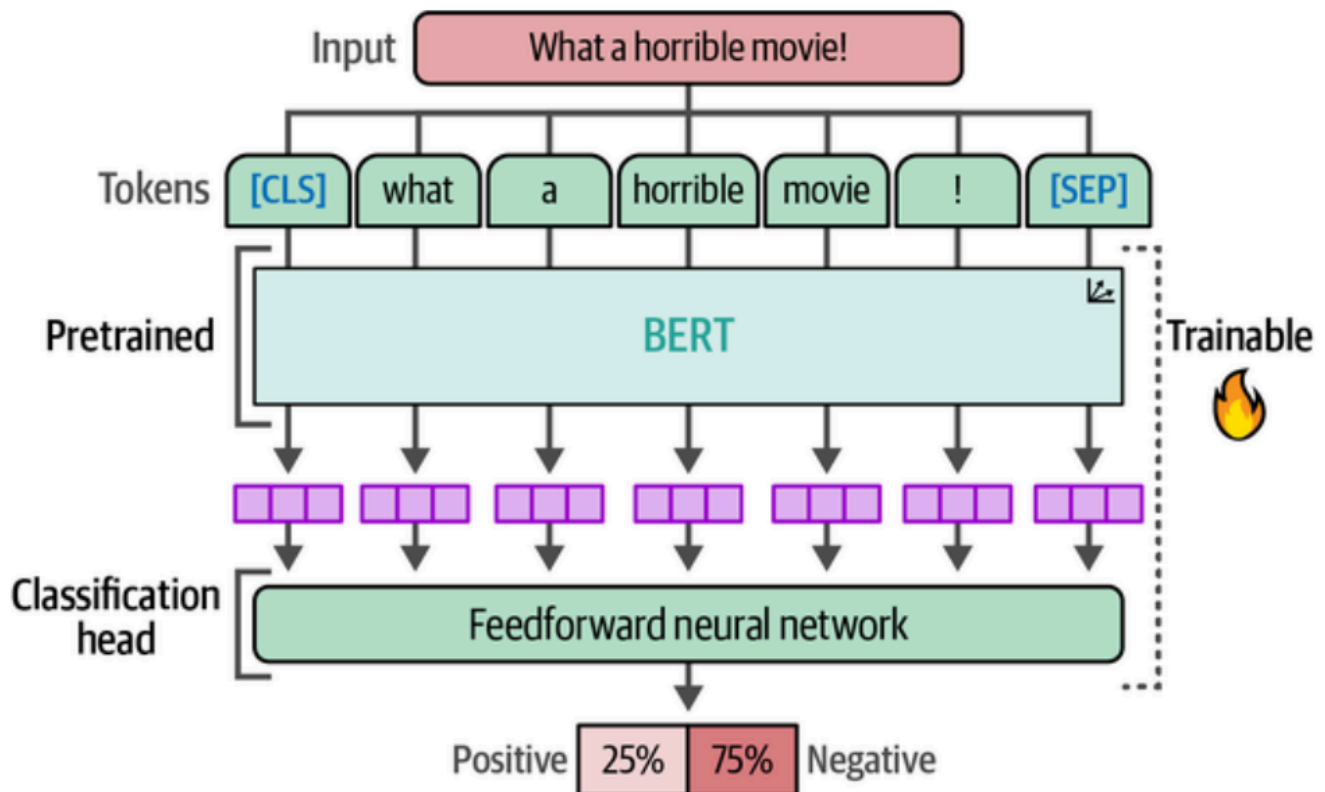


Figure 11-3. The architecture of a task-specific model. It contains a pretrained representation model (e.g., BERT) with an additional classification head for the specific task.

This means that the classification head and the pretrained model are both updated during the training. This means that they learn from one another and therefore be able to generate more

accurate representations (embeddings).

Fine-Tuning a Pretrained BERT Model

We will use the same dataset, the **Rotten Tomatoes** dataset, which contains 5,331 positive and 5,331 negative movie reviews from Rotten Tomatoes.

```
from datasets import load_dataset

# Prepare data and splits
tomatoes = load_dataset("rotten_tomatoes")
train_data, test_data = tomatoes["train"], tomatoes["test"]
```

As mentioned above, we use `bert-base-cased` as the pretrained model. This model was pretrained on the English Wikipedia and a large dataset of unpublished books.

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification

# Load model and tokenizer
model_id = "bert-base-cased"
model = AutoModelForSequenceClassification.from_pretrained(
    model_id, num_labels=2
)

tokenizer = AutoTokenizer.from_pretrained(model_id)
```

Now that the model and the tokenizer have been loaded, we will tokenize the data:

```
from transformers import DataCollatorWithPadding

# Pad to the longest sequence in the batch
data_collator = DataCollatorWithPadding(tokenizer=tokenizer)

def preprocess_function(examples):
    """ Tokenize input data """
    return tokenizer(examples["text"], truncation=True)

# Tokenize train/test data
tokenized_train = train_data.map(preprocess_function, batched=True)
tokenized_test = test_data.map(preprocess_function, batched=True)
```

Before creating the `Trainer`, we will want to prepare a special `DataCollator`.

A **DataCollator** is a special class that helps us build batches of data but also allows us to apply data augmentation.

To create equally sized representations, we use `DataCollatorWithPadding`.

```
import numpy as np
from datasets import load_metric

def compute_metrics(eval_pred):
    """Calculate F1 score"""
    logits, labels = eval_pred
    predictions = np.argmax(logits, axis=-1)

    load_f1 = load_metric("f1")
    f1 = load_f1.compute(predictions=predictions, references=labels)["f1"]
    return {"f1": f1}
```

With `compute_metrics` we can define any number of metrics that we are interested in and that can be printed out and logged during training. This is important to know if the model is overfitting.

Next, we instantiate the `Trainer`:

```
from transformers import TrainingArguments, Trainer

# Training arguments for parameter tuning
training_args = TrainingArguments(
    "model",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=1,
    weight_decay=0.01,
    save_strategy="epoch",
    report_to="none"
)

# Trainer which executes the training process
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_train,
    eval_dataset=tokenized_test,
    tokenizer=tokenizer,
    data_collator=data_collator,
```

```
        compute_metrics=compute_metrics,  
    )
```

The `TrainingArguments` class defines hyperparameters we want to tune, like the learning rate and the number of epochs.

```
trainer.evaluate()
```

```
{'eval_loss': 0.3663691282272339,  
 'eval_f1': 0.8492366412213741,  
 'eval_runtime': 4.5792,  
 'eval_samples_per_second': 232.791,  
 'eval_steps_per_second': 14.631,  
 'epoch': 1.0}
```

We get an F1 score of 0.85, higher than that of the task-specific model (0.80) from Chapter 4.

Freezing Layers

Using Hugging Face Transformers, we will freeze the main BERT model and allow training only for the classification head. This will allow for good comparisons.

```
# Load model and tokenizer  
model = AutoModelForSequenceClassification.from_pretrained(  
    model_id, num_labels=2  
)  
tokenizer = AutoTokenizer.from_pretrained(model_id)
```

By inspecting the layers of the model we can see which layers we might want to freeze:

```
# Print layer names  
for name, param in model.named_parameters():  
    print(name)
```

```
bert.embeddings.word_embeddings.weight  
bert.embeddings.position_embeddings.weight  
bert.embeddings.token_type_embeddings.weight  
bert.embeddings.LayerNorm.weight  
bert.embeddings.LayerNorm.bias  
bert.encoder.layer.0.attention.self.query.weight  
bert.encoder.layer.0.attention.self.query.bias  
...
```

```
bert.encoder.layer.11.output.LayerNorm.weight
bert.encoder.layer.11.output.LayerNorm.bias
bert.pooler.dense.weight
bert.pooler.dense.bias
classifier.weight
classifier.bias
```

There are 12 (0-11) encoder blocks consisting of attention heads, dense networks, and layer normalization.

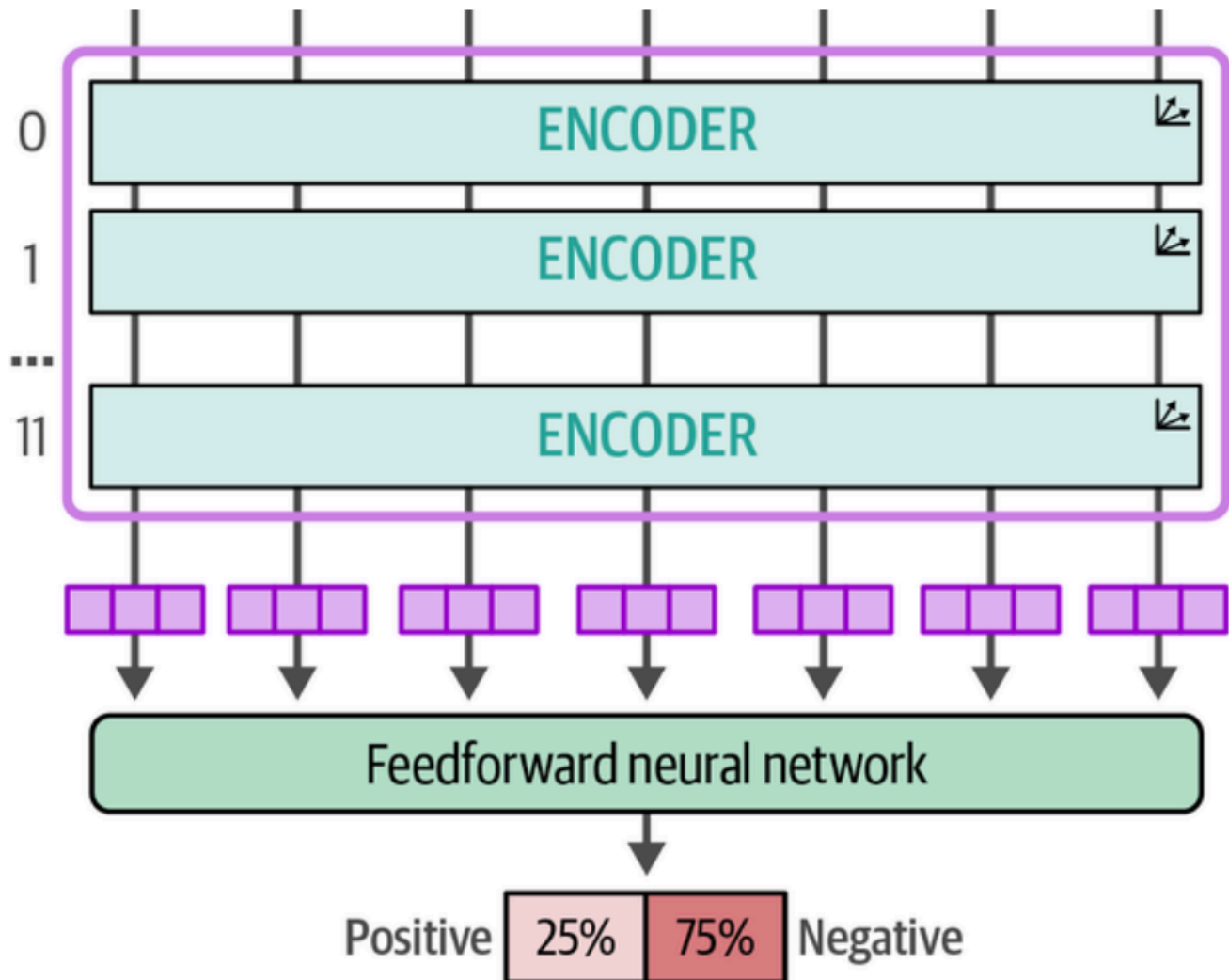


Figure 11-4. The basic architecture of BERT with the additional classification head.

We are going to freeze everything but the classification head, as in chapter 2.

```
for name, param in model.named_parameters():
    # Trainable classification head
    if name.startswith("classifier"):
        param.requires_grad = True
    # Freeze everything else
    else:
        param.requires_grad = False
```

This is illustrated by the following diagram:

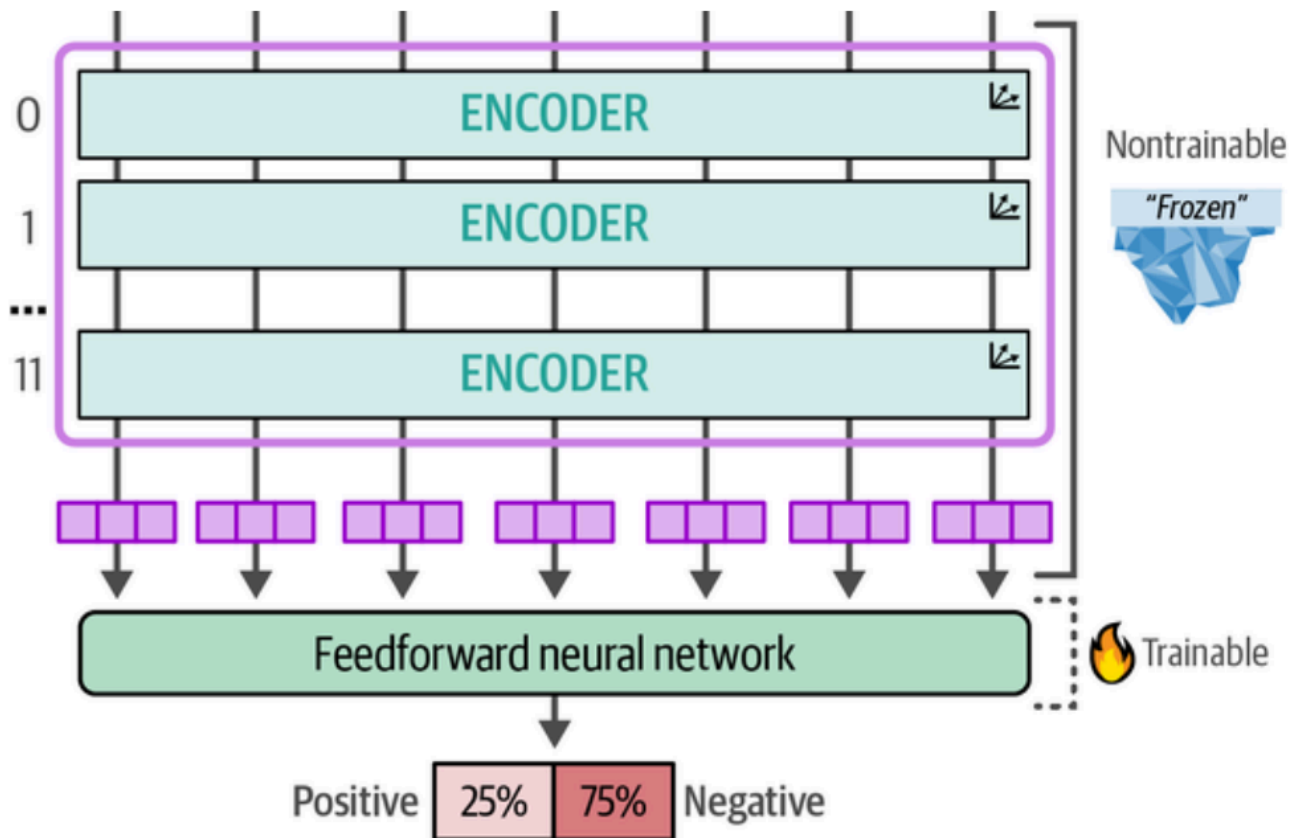


Figure 11-5. We fully freeze all encoder blocks and embedding layers such that the BERT model does not learn new representations during fine-tuning.

After freezing the model, we can start with the training:

```
from transformers import TrainingArguments, Trainer

# Trainer which executes the training process
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_train,
    eval_dataset=tokenized_test,
    data_collator=data_collator,
    compute_metrics=compute_metrics,
)
trainer.train()
```

Let's evaluate the model after training of 1 epoch:

```
trainer.evaluate()
```

```
{'eval_loss': 0.6821751594543457,  
'eval_f1': 0.6331058020477816,  
'eval_runtime': 4.0175,  
'eval_samples_per_second': 265.337,  
'eval_steps_per_second': 16.677,  
'epoch': 1.0}
```

As you can see, we get an F1 score of 0.63.

So instead of freezing all layers of the model, let's freeze everything up until encoder block 10 and see how this affects performance. This is illustrated below:

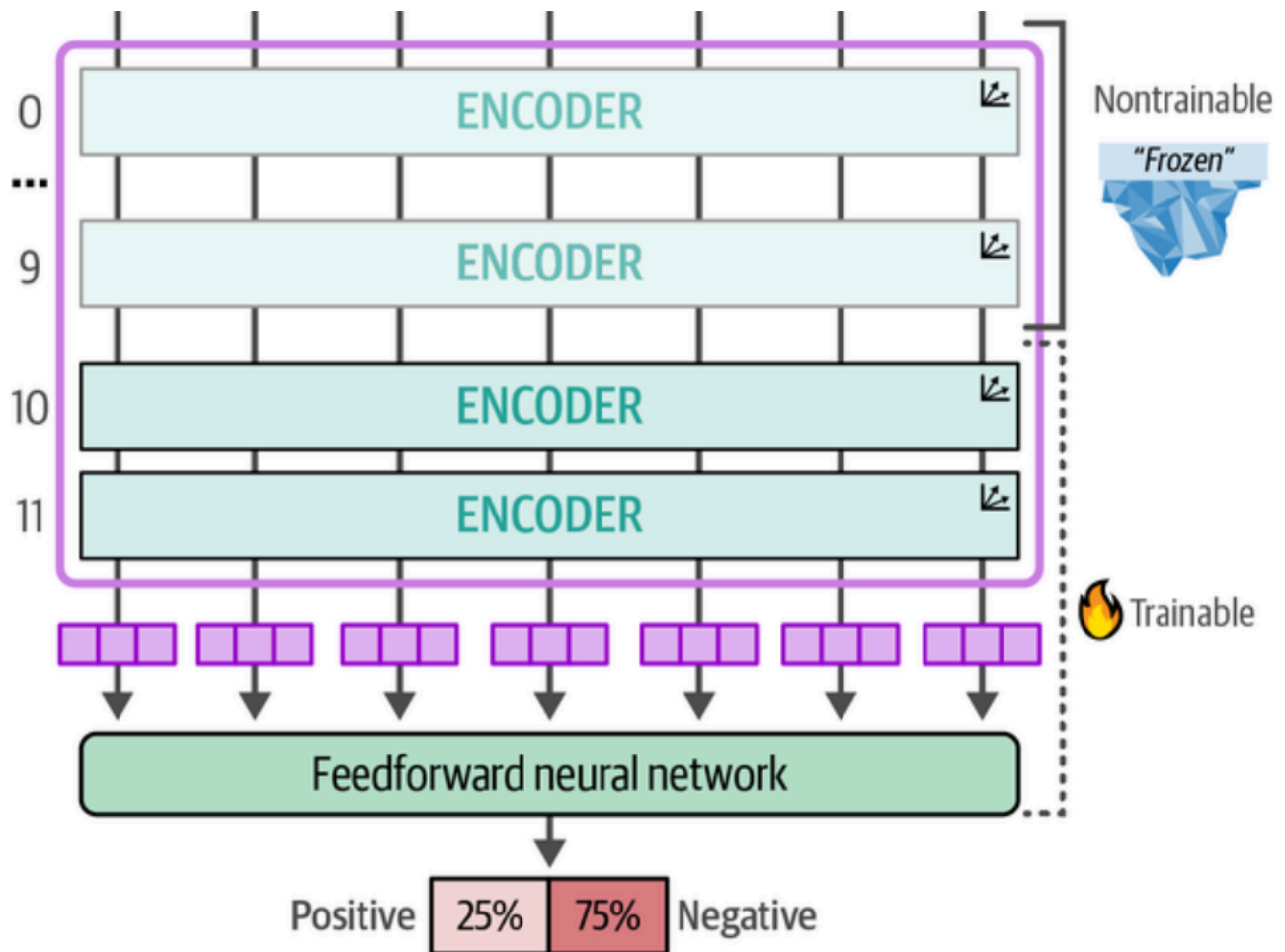


Figure 11-6. We freeze the first 10 encoder blocks of our BERT model. Everything else is trainable and will be fine-tuned.

The code to do that is as follows:

```
# Load model  
model_id = "bert-base-uncased"  
model = AutoModelForSequenceClassification.from_pretrained(  
    model_id, num_labels=2  
)
```



```

tokenizer = AutoTokenizer.from_pretrained(model_id)

# Encoder block 11 starts at index 165
# We freeze everything before that block
for index, (name, param) in enumerate(model.named_parameters()):
    if index < 165:
        param.requires_grad = False

# Trainer which executes the training process
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_train,
    eval_dataset=tokenized_test,
    tokenizer=tokenizer,
    data_collator=data_collator,
    compute_metrics=compute_metrics,
)
trainer.train()

```

Let's evaluate the results:

```

trainer.evaluate()

```

```

{'eval_loss': 0.40812647342681885,
 'eval_f1': 0.8,
 'eval_runtime': 3.7125,
 'eval_samples_per_second': 287.137,
 'eval_steps_per_second': 18.047,
 'epoch': 1.0}

```

The F1 score this time is 0.8, much higher than the previous 0.63 when freezing all the layers.

This shows that training less is sufficient if the necessary computing power is available.

Furthermore, this can be seen in the graph below, showing that training only the first five encoder blocks is enough to almost reach the performance when all encoder blocks are trained.

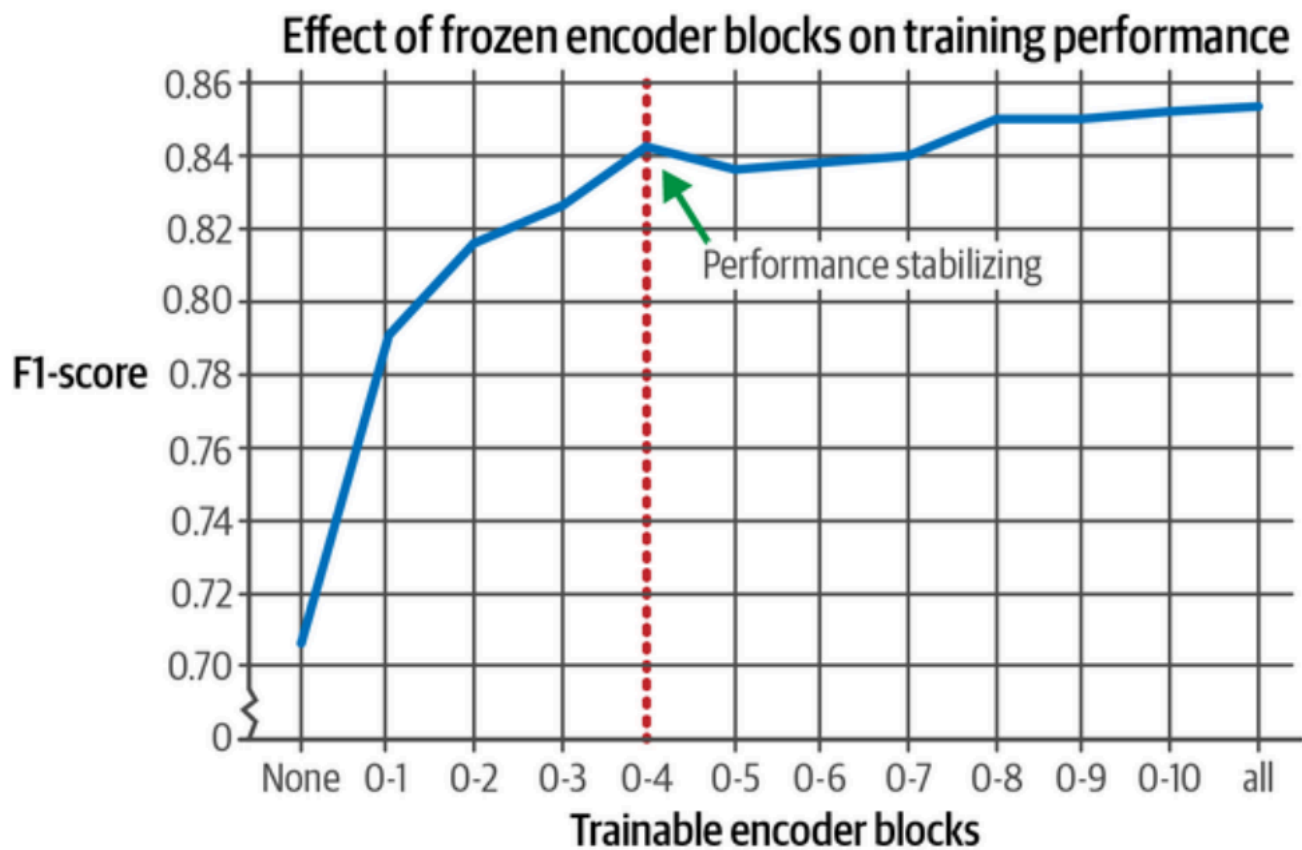


Figure 11-7. The effect of freezing certain encoder blocks on the performance of the model. Training more blocks leads to improved performance but stabilizes early on.

Few-Shot Classification

Few-shot classification is a technique within supervised classification where the classifier learns target labels based on only a few labeled examples.

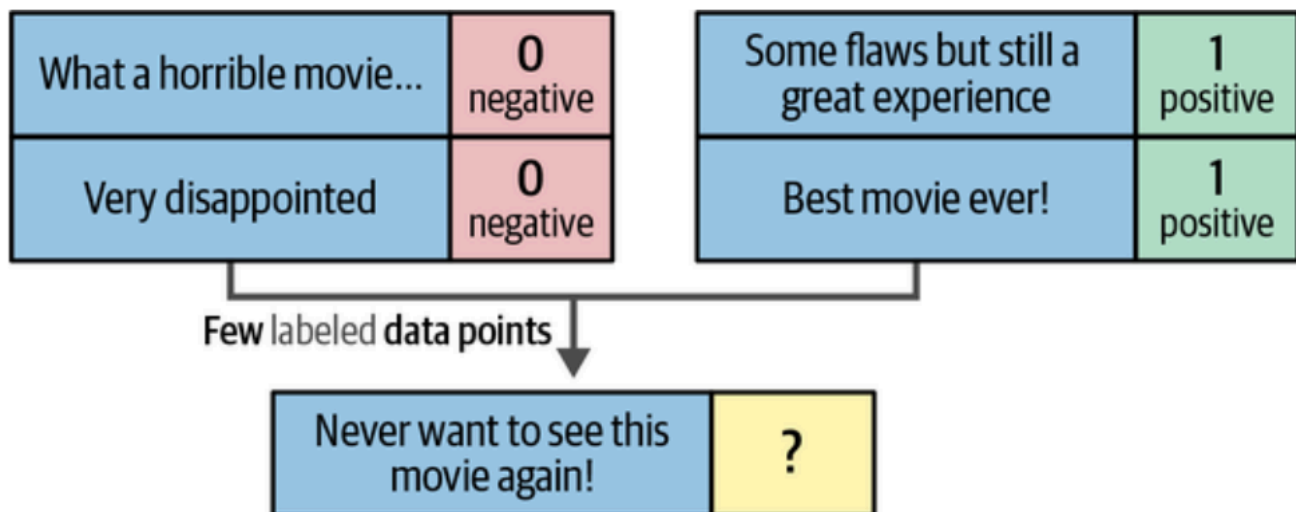


Figure 11-8. In few-shot classification, we only use a few labeled data points to learn from.

SetFit: Efficient Fine-Tuning with Few Training Examples

SetFit is built on top of the `sentence-transformers` architecture to generate high quality textual representations that are updated during training.

The algorithm of SetFit consists of **3 steps**:

1. Sampling training data

Based on in-class and out-class selection of labeled data it generates positive (similar) and negative (dissimilar) pairs of sentences

2. Fine-tuning embeddings

Fine-tuning a pretrained embedding model based on previously generated training data

3. Training a classifier

Create a classification head on top of the embedding model and train it using the previously generated training data

Before fine-tuning an embedding model, we need to generate training data. The model assumes the training data to be samples of positive (similar) and negative (dissimilar) pairs of sentences. But the input data we have is generally not labeled as such.

For example, let's say we have the training dataset that classifies text into 2 categories: text about programming languages, and text about pets.

Text	Class
I write my code in Python	Programming languages
I should practice SQL	Programming languages
My dog is a labrador	Pets
I have a Siamese cat	Pets

Figure 11-9. Data in two classes: text about programming languages and text about pets.

In **step 1**, SetFit handles this by pairing all sentences of one class with those of the other. So imagine you have 16 sentences about sports, we can create $16 \cdot (16 - 1) / 2 = 120$ pairs that we label as positive pairs.

We can use the same process to generate negative (dissimilar) pairs by collecting pairs from different classes.

Text 1	Text 2	Pair type
I write my code in Python	I should practice SQL	Positive
My dog is a labrador	I have a Siamese cat	Positive
I write my code in Python	My dog is a labrador	Negative
I have a Siamese cat	I should practice SQL	Negative

Figure 11-10. Step 1: sampling training data. We assume sentences within a class are similar and create positive pairs while sentences in different classes become negative pairs.

In **step 2**, we can use the generated sentence pairs to fine-tune the embedding model.

This uses the method called **contrastive learning** to fine-tune a pretrained model, as discussed in Chapter 10. This is illustrated below:

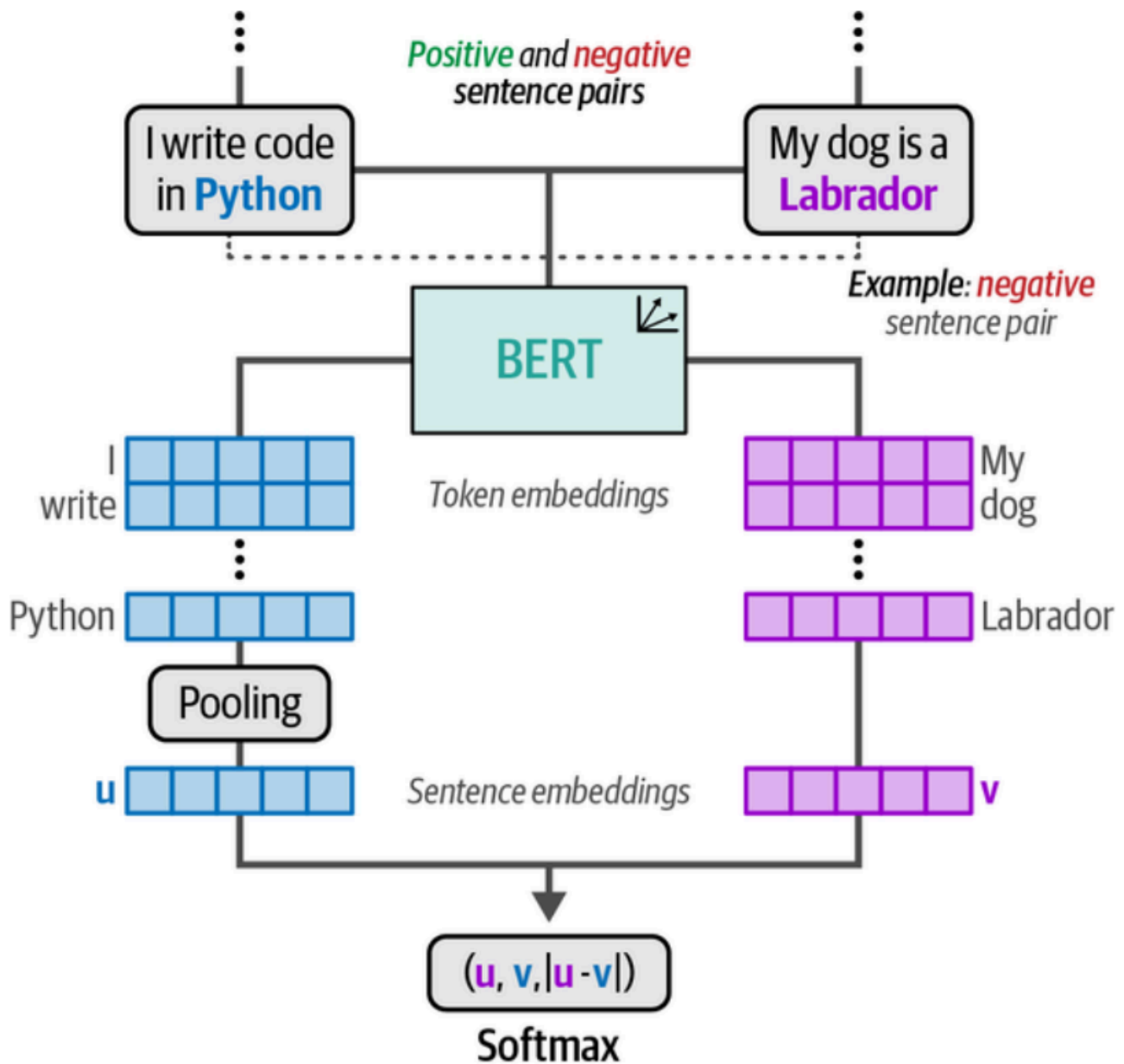


Figure 11-11. Step 2: Fine-tuning a *SentenceTransformers* model. Using contrastive learning, embeddings are learned from positive and negative sentence pairs.

The goal of fine-tuning this embedding model is that it can create embeddings that are turned to the classification task. The relevance of the classes, and their relative meaning, are distilled into the embeddings through fine-tuning the embedding model.

In **step 3**, we generate embeddings for all sentences and use those as the input of a classifier.

We use the `sentence-transformers` model we finetuned to convert the sentences into embeddings.

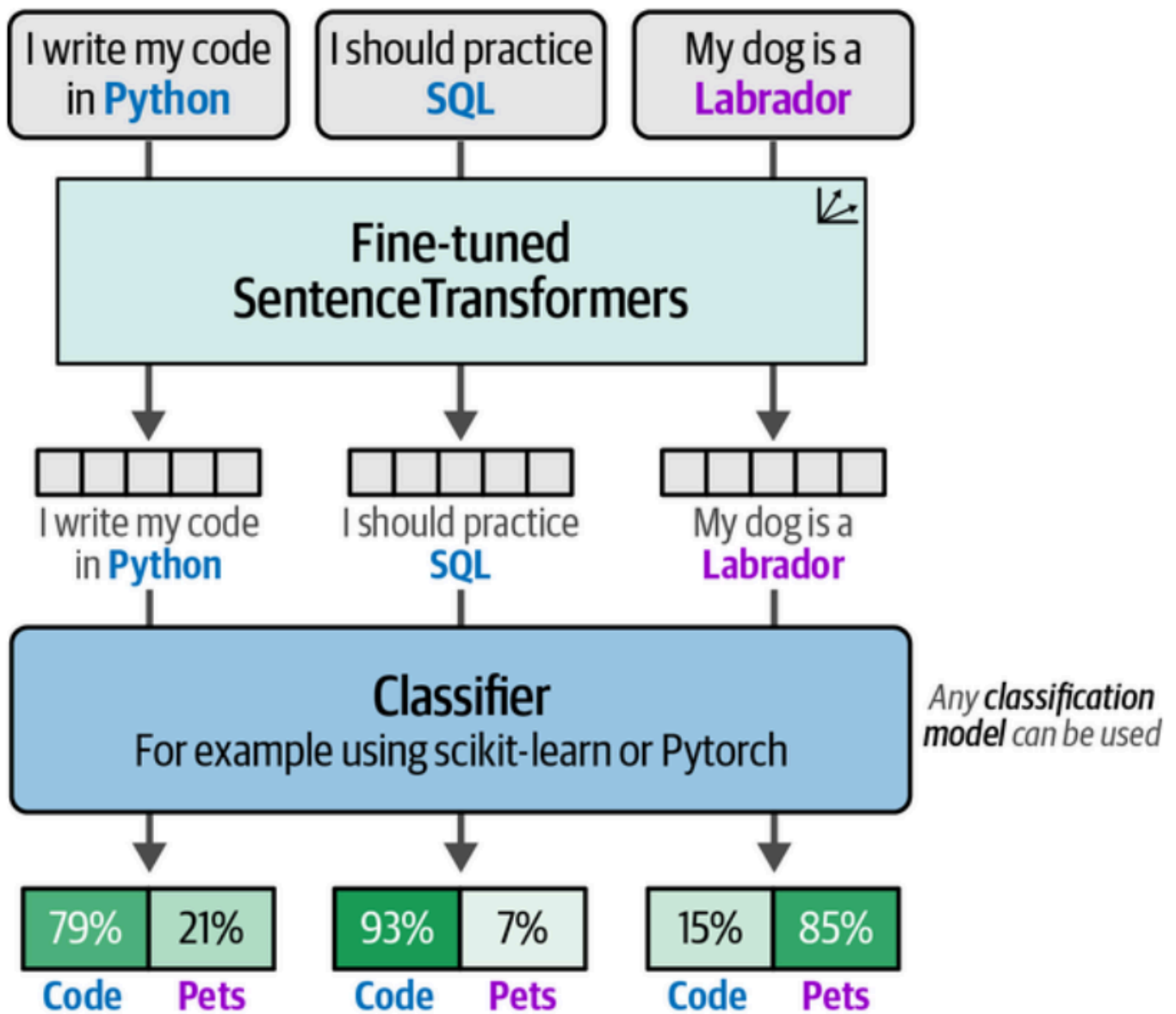


Figure 11-12. Step 3: Training a classifier. The classifier can be any scikit-learn model or a classification head.

The three main steps of SetFit is illustrated in the diagram:

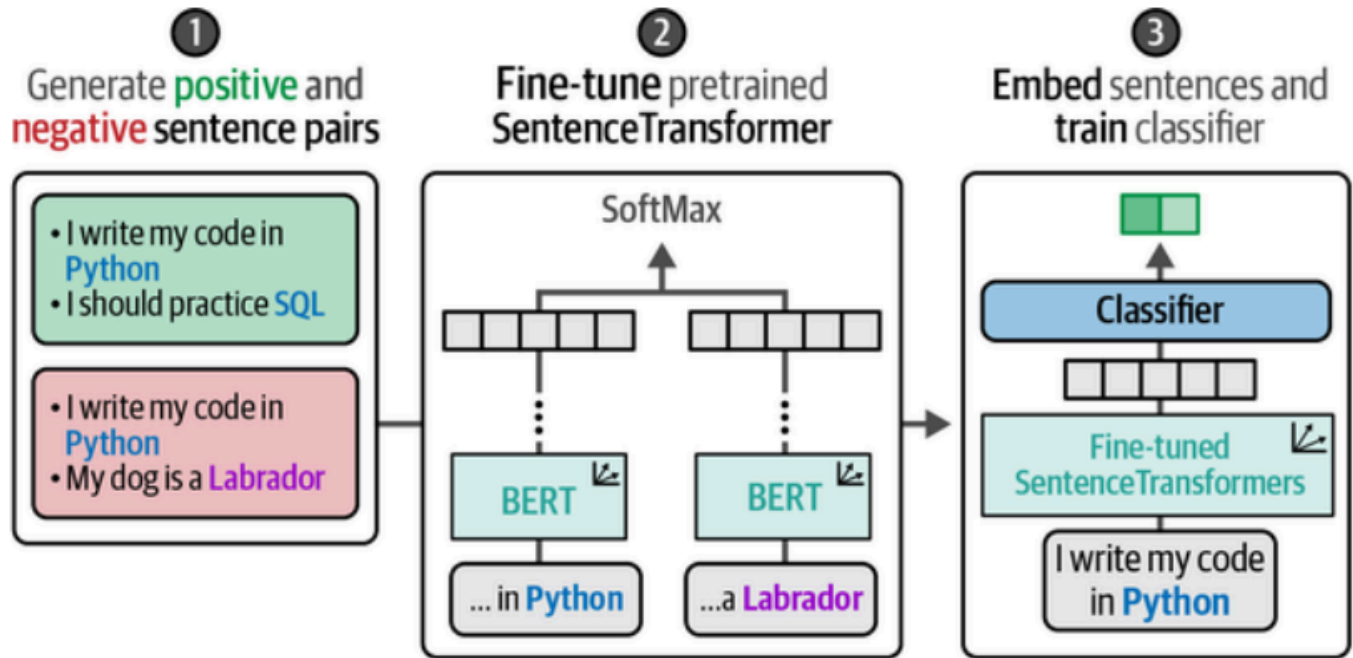


Figure 11-13. The three main steps of SetFit.

Fine-Tuning for Few-Shot Classification

Since this is few-shot, we only use 16 examples per class (32 in total) of the 8,500 movie reviews used before.

```
from setfit import sample_dataset

# We simulate a few-shot setting by sampling 16 examples per class

sampled_train_data = sample_dataset(tomatoes["train"], num_samples=16)
```

After loading the pretrained `SentenceTransformer` model, we define the `SetFitTrainer`.

A logistic regression model is chosen as the classifier to train by default.

```
from setfit import TrainingArguments as SetFitTrainingArguments
from setfit import Trainer as SetFitTrainer

# Define training arguments
args = SetFitTrainingArguments(
    num_epochs=3, # the number of epochs to use for contrastive learning
    num_iterations=20
)
args.eval_strategy = args.evaluation_strategy

#Create trainer
```

```
trainer = SetFitTrainer(  
    model=model,  
    args=args,  
    train_dataset=sampled_train_data,  
    eval_dataset=test_data,  
    metric="f1"  
)
```

We only need to call `train` to start the training loop. When we do, we should get the following output:

```
# Training loop  
  
trainer.train()
```

```
***** Running training *****  
Num unique pairs = 1280  
Batch size = 16  
Num epochs = 3  
Total optimization steps = 240
```

Notice that the number of unique pairs in the output is 1,280. This is because, by default, 20 sentence pair combinations are generated for each sample in data. So $20 * 32 = 680$ multiplied by 2 (for negative and positive pairs).

Let's now evaluate the model:

```
# Evaluate the model on our test data  
trainer.evaluate()
```

```
{'f1': 0.8363988383349468}
```

Continued Pretraining with Masked Language

So far, we have used a pretrained model and finetuned it.

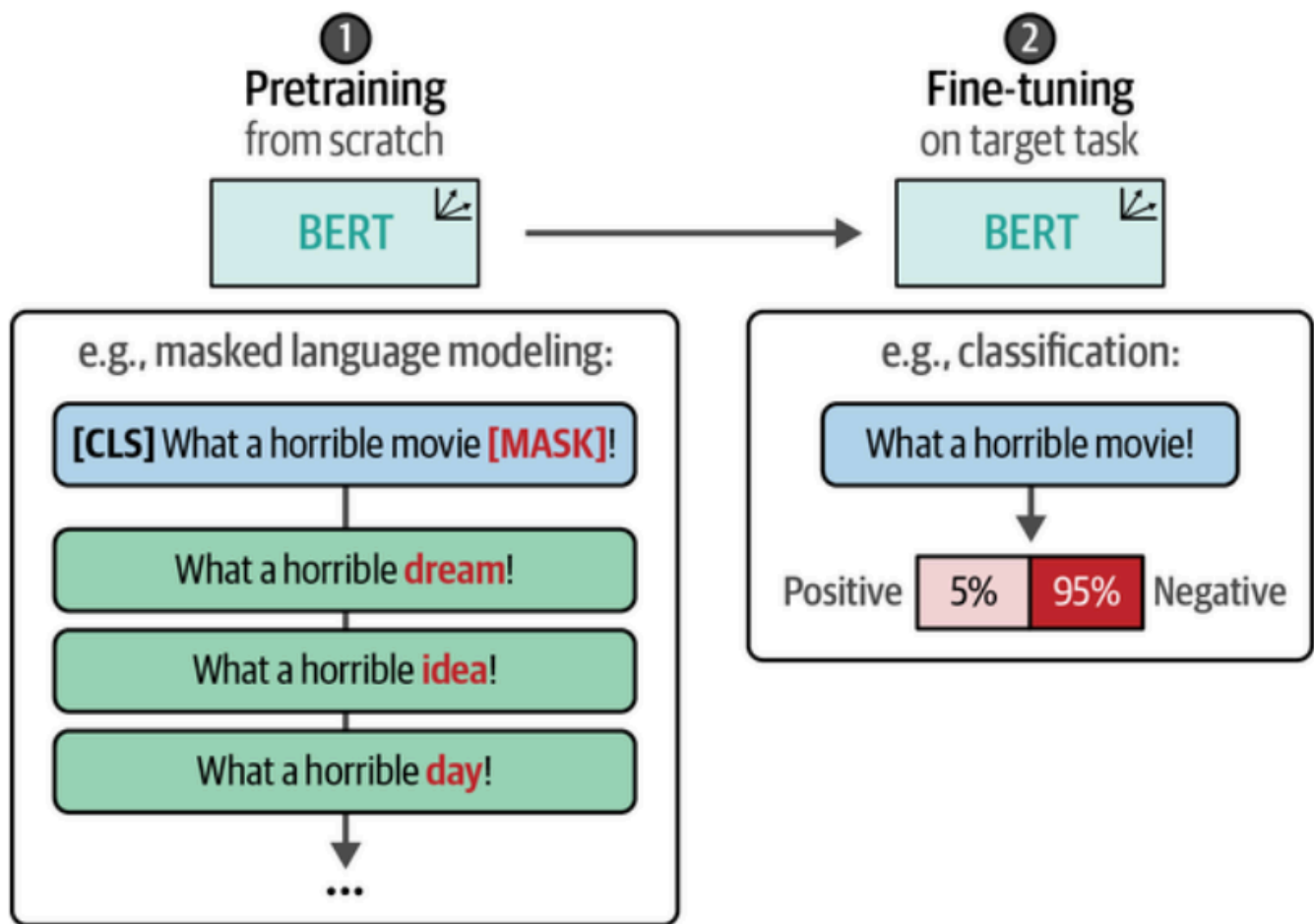


Figure 11-14. To fine-tune the model on a target task—for example, classification—we either start with pretraining a BERT model or use a pretrained one.

This is a **two-step process**:

1. Pretraining a model (already done for us)
2. Fine-tuning it for a particular task

This two-step process is used throughout many applications. However, since these models are pretrained on very general data (e.g., Wikipedia), it is not tuned to domain-specific words.

To address this, an extra step is added between these two steps. It is **continue pretraining** an already pretrained BERT model.

We simply continue training the BERT model using **masked language modeling (MLM)**.

The data used to train it are domain-specific.

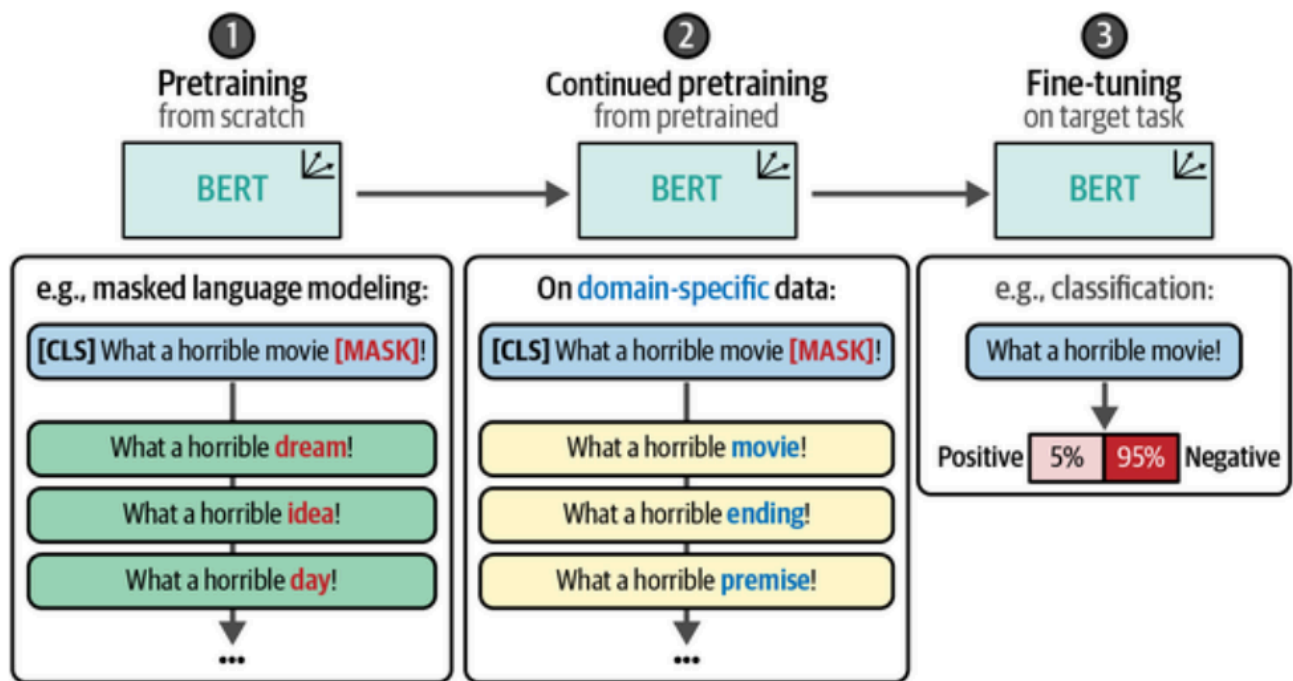


Figure 11-15. Instead of a two-step approach, we can add another step that continues to pretrain the pretrained model before fine-tuning it on the target task. Notice how the masks were filled with abstract concepts in 1 while they were filled with movie-specific concepts in 2.

The advantage of this is that it doesn't require pretraining from scratch. This is illustrated in the following diagram:

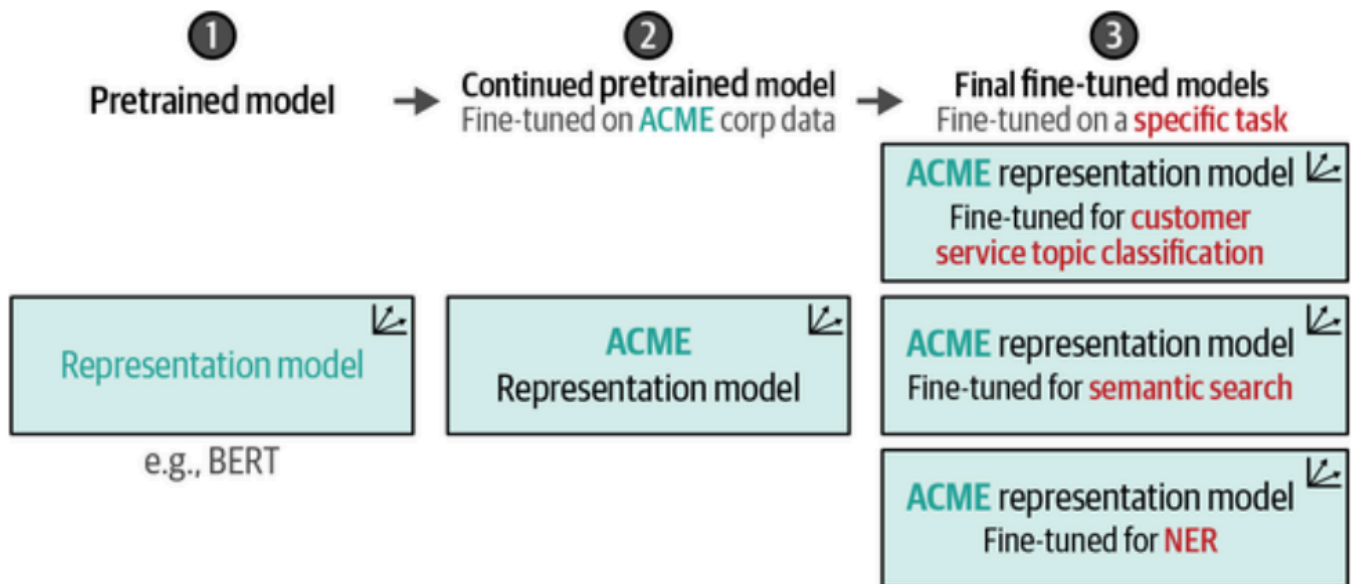


Figure 11-16. The three-step approach illustrated for specific use cases.

in the following example, we see how the 'step 2' is applied.

We start by loading the `bert-base-cased` model:

```
from transformers import AutoTokenizer, AutoModelForMaskedLM

# Load model for masked language modeling (MLM)
```

```
model = AutoModelForMaskedLM.from_pretrained("bert-base-cased")
tokenizer = AutoTokenizer.from_pretrained("bert-base-cased")
```

We need to tokenize the raw sentences and remove the labels since this is not a supervised task:

```
def preprocess_function(examples):
    return tokenizer(examples["text"], truncation=True)

# Tokenize data
tokenized_train = train_data.map(preprocess_function, batched=True)
tokenized_train = tokenized_train.remove_columns("label")
tokenized_test = test_data.map(preprocess_function, batch=True)
tokenized_test = tokenized_test.remove_columns("label")
```

Previously, we used `DataCollatorWithPadding`, which dynamically pads the input it receives.

Now, we use `DataCollator` that will perform the masking of tokens for us.

There are generally **two methods** of masking used:

1. **Token masking**: randomly mask 15% of the tokens in a sentence. A part of a word could be masked.
2. **Whole-word masking**: To avoid masking just a part of a word, we use *whole-word masking*.

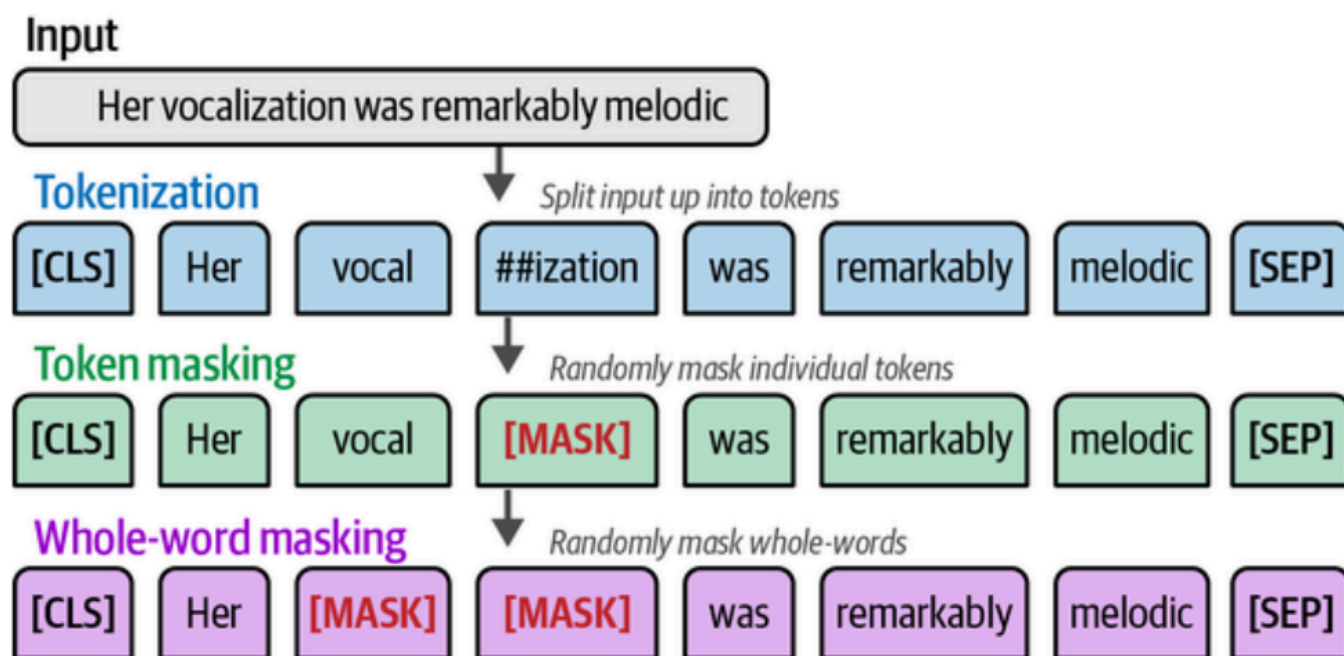


Figure 11-17. Different methods for randomly masking tokens.

Generally, predicting whole words tends to be more complicated than tokens, which makes the model perform better as it needs to learn more accurate and precise representations during training.

We will be going with token masking and use `DataCollatorForLanguageModeling` for faster convergence.

For whole-word masking, you can replace it with `DataCollatorForWholeWordMask`.

Lastly, we set the percentage of masked tokens to 15%:

```
from transformers import DataCollatorForWholeWordMask

# Masking tokens
data_collator = DataCollatorForLanguageModeling(
    tokenizer=tokenizer,
    mlm=True,
    mlm_probability=0.15,
)
```

Next, we will create the `Trainer` for running the MLM task and specify certain parameters:

```
# Training arguments for parameter tuning
training_args = TrainingArguments(
    "model",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=10,
    weight_decay=0.01,
    save_strategy="epoch",
    report_to="none",
)
```

Initialize the trainer:

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_train,
    eval_dataset=tokenized_test,
    tokenizer=tokenizer,
```

```
data_collator=data_collator
)
```

After we *continue pretraining*, we save the pretrained tokenizer.

```
# Save the pre-trained tokenizer
tokenizer.save_pretrained("mlm")

# Train model
trainer.train()

# Save updated model
model.save_pretrained("mlm")
```

This gives us an updated model in the *mlm* folder. To evaluate its performance we would normally fine-tune the model on a variety of tasks. For this context, however, we will run some masking tasks to see if it has learned from its continued training.

```
from transformers import pipeline

# Load and create predictions
mask_filler = pipeline("fill-mask", model="bert-base-cased")
preds = mask_filler("What a horrible [MASK]!")

# Print results
for pred in preds:
    print(f">>> {pred['sequence']}")
```

```
>>> What a horrible idea!
>>> What a horrible dream!
>>> What a horrible thing!
>>> What a horrible day!
>>> What a horrible thought!
```

Let's see what the updated model predicts:

```
from transformers import pipeline

# Load and create predictions
mask_filler = pipeline("fill-mask", model="mlm")
preds = mask_filler("What a horrible [MASK]!")

# Print results
```

```
for pred in preds:  
    print(f">>> {pred['sequence']}")
```

```
>>> What a horrible movie!  
>>> What a horrible film!  
>>> What a horrible mess!  
>>> What a horrible comedy!  
>>> What a horrible story!
```

As you can see, the model now generates words from the data we trained it on (Rotten Tomatoes reviews dataset).

Named-Entity Recognition

Unlike typical classification of entire documents or texts, **Named-Entity Recognition** involves identifying tokens and/or words of entities (names, places, events, etc.).

This is helpful for de-identification and anonymization tasks when there are sensitive data.

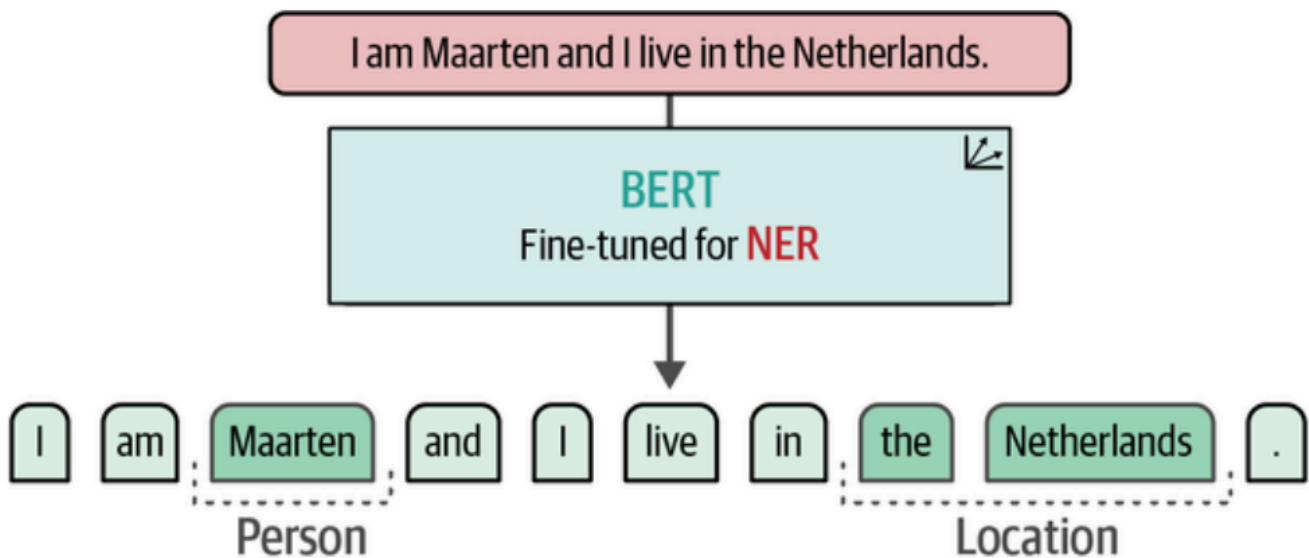


Figure 11-18. Fine-tuning a BERT model for NER allows for the detection of named entities, such as people or locations.

Fine-tuning the pretrained BERT model follows a similar architecture to what we observed with document classification.

Unlike classification where an entire document is classified, NER uses word-level classification. This classification is not of entire words but of tokens that make those words.

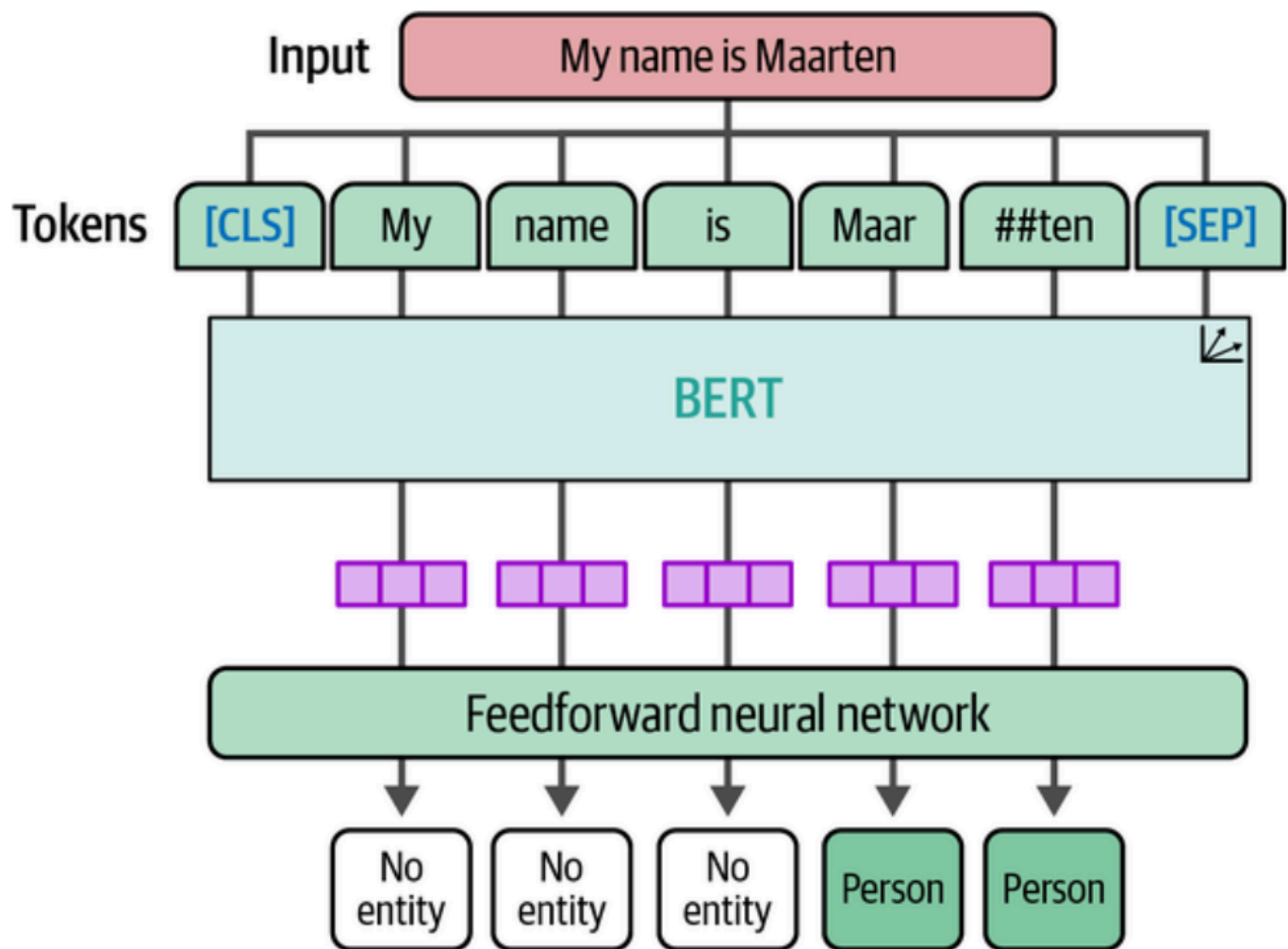


Figure 11-19. During the fine-tuning process of a BERT model, individual tokens are classified instead of words or entire documents.

Preparing Data for Named-Entity Recognition

In this example, we will use the English version of the `CoNLL-2003` dataset, which contains several different types of **named entities** (person, organizations, etc.) and has roughly 14,000 training samples.

```
# The CoNLL-2003 dataset for NER
dataset = load_dataset("conll", trust_remote_code=True)
```

Let's inspect the structure of the data with an example:

```
example = dataset["train"][848]
example
```

```
{'id': '848',
 'tokens': ['Dean',
```

```

'Palmer',
'hit',
'his',
'30th',
'homer',
'for',
'the',
'Rangers',
'.'],
'pos_tags': [22, 22, 38, 29, 16, 21, 15, 12, 23, 7],
'chunk_tags': [11, 12, 21, 11, 12, 12, 13, 11, 12, 0],
'ner_tags': [1, 2, 0, 0, 0, 0, 0, 0, 3, 0]}

```

This dataset provides us with labels for each word given in a sentence. These labels can be found in the `ner_tags` key:

```

label2id = {
    "O": 0, "B-PER": 1, "I-PER": 2, "B-ORG": 3, "I-ORG": 4,
    "B-LOC": 5, "I-LOC": 6, "B-MISC": 7, "I-MISC": 8
}

id2label = {index: label for label, index in label2id.items()}
label2id

```

```

{'O': 0,
 'B-PER': 1,
 'I-PER': 2,
 'B-ORG': 3,
 'I-ORG': 4,
 'B-LOC': 5,
 'I-LOC': 6,
 'B-MISC': 7,
 'I-MISC': 8}

```

These entities correspond to specific categories: a person (PER), organization (ORG), location (LOC), miscellaneous entities (MISC), and no entity (O). They are prefixed with either a B (beginning) or an I (inside).

If two tokens that follow each other are part of the same phrase, then the start of that phrase is indicated with B, which is followed by an I to show that they belong to each other and are not independent entities.

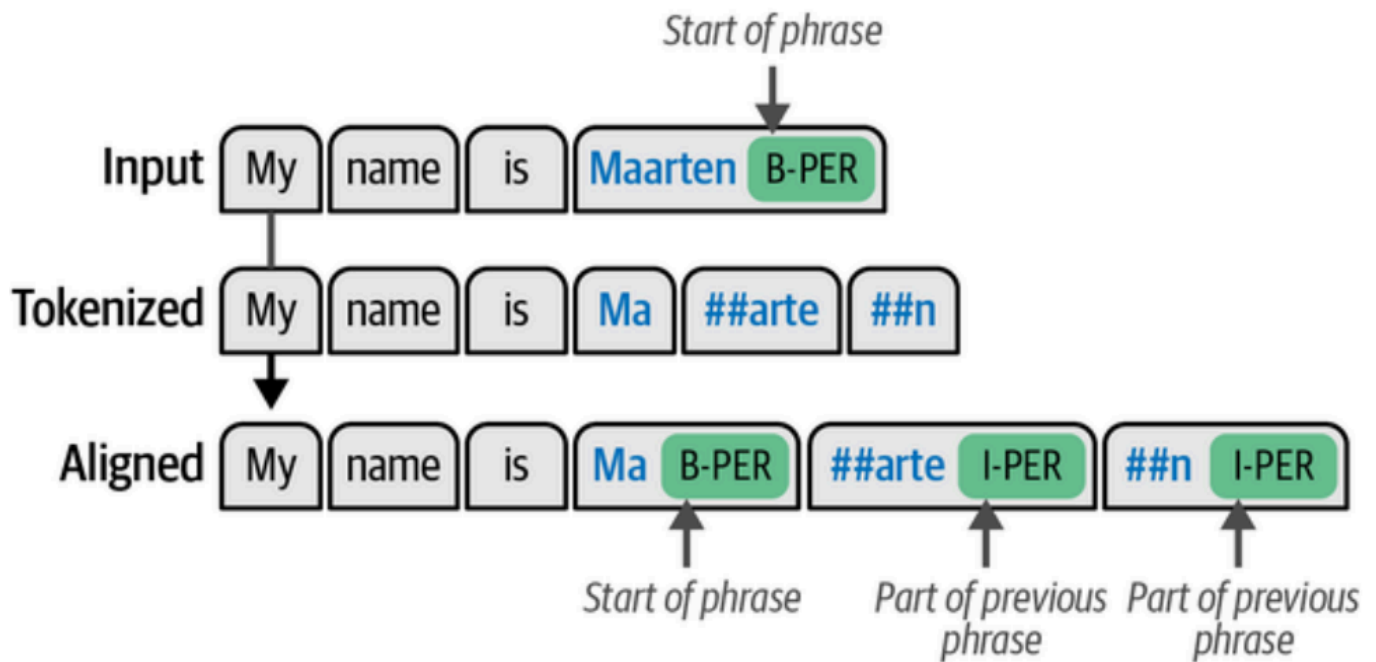


Figure 11-21. The alignment process of labeling tokenized input.

We now tokenize the words with the tokenizer of the pretrained model `bert-base-cased`:

```
from transformers import AutoModelForTokenClassification

# Load tokenizer
tokenizer = AutoTokenizer.from_pretrained("bert-base-cased")

# Load model
model = AutoModelForTokenClassification.from_pretrained(
    "bert-base-cased",
    num_labels=len(id2label),
    id2label=id2label,
    label2id=label2id
)
```

Let's explore how the tokenizer would process our example:

```
# Split individual tokens into sub-tokens
token_ids = tokenizer(example["tokens"], is_split_into_words=True)
["input_ids"]

sub_tokens = tokenizer.convert_ids_to_tokens(token_ids)
sub_tokens
```

```
['[CLS]',
 'Dean',
```

```
'Palmer',  
'hit',  
'his',  
'30th',  
'home',  
'##r',  
'for',  
'the',  
'Rangers',  
'.',  
'[SEP]'
```

The tokenizer added the `[CLS]` and `[SEP]` tokens as we learned in Chapters 2 and 3.

Note that the word 'homer' was further split up into the tokens 'home' and '##r'.

This creates a bit of a problem for us since we have labeled data at the word level but not at the token level.

This can be resolved by **aligning the labels with their subtoken counterparts during tokenization**.

Let's consider the word 'Maarten', which has the label **B-PER** to signal that this is a person.

If we pass that word through the tokenizer, it splits the word up into the tokens 'Ma', '##arte', and '##n'.

We cannot use the B-PER entity for all tokens as that would signal that the three tokens are all independent people.

Whenever an entity is split into tokens, the first token should have **B** (for beginning) and the following should be **I** (for inner).

Therefore, 'Ma' will get the **B-PER** to signal the start of a phrase, and '##arte', and '##n' will get the **I-PER** to signal they belong to a phrase.

This alignment process is illustrated below.

We create a function, `align_labels`, that will tokenize the input and align these tokens with their updated labels during tokenization:

```
def align_labels(examples):  
    token_ids = tokenizer(  
        examples["tokens"],  
        truncation=True,  
        is_split_into_words=True
```

```

)

labels = examples["ner_tags"]
updated_labels = []

for index, label in enumerate(labels):
    # Map tokens to their respective word
    word_ids = token_ids.word_ids(batch_index=index)
    previous_word_idx = None
    label_ids = []

    for word_idx in word_ids:
        # The start of a new word
        if word_idx != previous_word_idx:
            previous_word_idx = word_idx
            updated_label = -100 if word_idx is None else
label[word_idx]
            label_ids.append(updated_label)

        # Special token is -100
        elif word_idx is None:
            label_ids.append(-100)

        # If the label is B-XXX we change it to I-XXX
        else:
            updated_label = label[word_idx]
            if updated_label % 2 == 1:
                updated_label += 1
            label_ids.append(updated_label)

    updated_labels.append(label_ids)

token_ids["labels"] = updated_labels
return token_ids

tokenized = dataset.map(aligned_labels, batched=True)

```

Looking at our example, note that additional labels (-100) were added for the [CLS] and [SEP] tokens:

```

# Difference between original and updated labels
print(f"Original: {example['ner_tags']}")
print(f"Updated: {tokenized['train'][848]['labels']}")

```

```
Original: [1, 2, 0, 0, 0, 0, 0, 0, 0, 3, 0]
Updated: [-100, 1, 2, 0, 0, 0, 0, 0, 0, 0, 3, 0, -100]
```

Now that we have tokenized and aligned the labels, we can start thinking about defining our evaluation metrics.

This is also different from what we have seen before.

Instead of a single prediction per document, we now have **multiple predictions per document**, namely per token.

We will make use of the `evaluate` package by Hugging Face to create a `compute_metrics` function that allows us to evaluate performance on a token level:

```
import evaluate

# Load sequential evaluation
segeval = evaluate.load("segeval")

def compute_metrics(eval_pred):
    # Create predictions
    logits, labels = eval_pred
    predictions = np.argmax(logits, axis=-1)

    true_predictions = []
    true_labels = []

    # Document-level iteration
    for prediction, label in zip(predictions, labels):

        # Token-level iteration
        for token_prediction, token_label in zip(prediction, label):

            # We ignore special tokens
            if token_label != -100:
                true_predictions.append([id2label[token_prediction]])
                true_labels.append([id2label[token_label]])

    results = segeval.compute(
        predictions=true_predictions, references=true_labels
    )

    return {"f1": results["overall_f1"]}
```

Fine-Tuning for Named-Entity Recognition

We are nearly there.

Instead of `DataCollatorWithPadding`, we need a collator that works with classification on a token level, namely `DataCollatorForTokenClassification`:

```
from transformers import DataCollatorForTokenClassification

# Token-classification DataCollator
data_collator = DataCollatorForTokenClassification(tokenizer=tokenizer)
```

Now that we have loaded our model, the rest of the steps are similar to previous training procedures in this chapter.

We define a trainer with specific arguments that we can tune and create a `Trainer`:

```
# Training arguments for parameter tuning
training_args = TrainingArguments(
    "model",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=1,
    weight_decay=0.01,
    save_strategy="epoch",
    report_to="none"
)

# Initialize Trainer
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized["train"],
    eval_dataset=tokenized["test"],
    tokenizer=tokenizer,
    data_collator=data_collator,
    compute_metrics=compute_metrics,
)

trainer.train()
```

We then evaluate the model that we created:

```
# Evaluate the model on our test data
trainer.evaluate()
```

Lastly, let's save the model and use it in a pipeline for inference.

This allows us to check certain data so we can manually inspect what happens during inference and if we are satisfied with the output:

```
from transformers import pipeline

# Save our fine-tuned model
trainer.save_model("ner_model")

# Run inference on the fine-tuned model
token_classifier = pipeline(
    "token-classification",
    model="ner_model",
)

token_classifier("My name is Maarten.")
```

```
[{'entity': 'B-PER',
'score': 0.99534035,
'index': 4,
'word': 'Ma',
'start': 11,
'end': 13},
{'entity': 'I-PER',
'score': 0.9928328,
'index': 5,
'word': '##arte',
'start': 13,
'end': 17},
{'entity': 'I-PER',
'score': 0.9954301,
'index': 6,
'word': '##n',
'start': 17,
'end': 18}]
```

In the sentence "My name is Maarten", the word "Maarten" and its subtokens were correctly identified as a person.